Implementacja algorytmu mnożenia liczb dowolnej precyzji

Cybulski Mateusz, Derron Tomasz, Domiński Bartosz

Streszczenie

Implementacja algorytmu mnożenia Toom-Cook'a (Toom-3) dla liczb całkowitych dowolnej precyzji w C++. Zaprojektowanie i przeprowadzenie eksperymentu mającego na celu porównanie utworzonej implementacji z istniejącymi bibliotekami: GMP i wbudowaną biblioteką w Python.

Spis treści

1 Metodologia **2**
1.1 Cel eksperymentu 2
1.2 Opis eksperymentu 2
1.3 Plan osiągnięcia celów projektowych 2

2 Generowanie próbki **4**

3 Implementacja w języku Python **4**
3.1 vector_to_int 4
3.2 choose_time_unit 4
3.3 main_func 4

4 Implementacja z wykorzystaniem biblioteki GNU MP **4**
4.1 vector_to_mpz 5
4.2 TimeResult 5
4.3 choose_time_unit 5
4.4 main_func 5

5 Analiza wyników **5**
5.1 load_file 5
5.2 equalize_units 5
5.3 remove_outliers 5
5.4 calculate_stats 5
5.5 plot_one_method 6
5.6 draw_regression 6
5.7 workflow 6

6 Cel dodatkowy **6**
6.1 Generowanie danych testowych 7
6.2 Wyniki eksperymentu 7

1 Metodologia

1.1 Cel eksperymentu

Celem eksperymentu jest zbadanie zależności pomiędzy czasem wykonania operacji mnożenia a rozmiarem danych wejściowych dla utworzonej implementacji algorytmu mnożenia dużych liczb. W drugiej części eksperymentu, na podstawie otrzymanej zależności, nastąpi porównanie naszej implementacji z istniejącymi bibliotekami.

1.2 Opis eksperymentu

Zbiór liczb użyty w eksperymencie będzie składał się z liczb w zapisie o podstawie $\beta = 2^{32}$. W eksperymencie zostaną użyte liczby z zakresu od liczb 50-słowowych do 1000-słowowych (zakładamy, że słowo = 32 bity).

Będziemy mierzyć czas wykonania operacji mnożenia dla liczb o długości 50, 100, 150, ... , 950, 1000 słów. Dla każdej z podanych długości zostanie wygenerowanych 40 liczb o zadanej długości, następnie w parach (20 par dla każdej z ustalonych długości) zostanie przeprowadzone mnożenie. Zakres długości liczb został wybrany na podstawie wykresu 1, pokazanym poniżej.

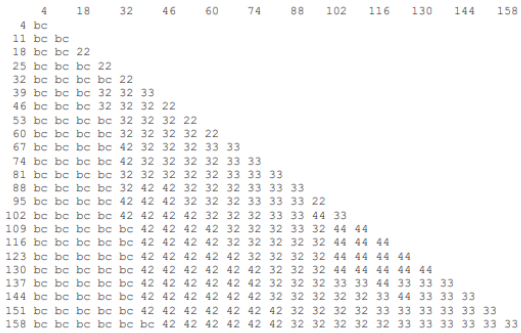


Figure 1.1 The best algorithm to multiply two numbers of x and y words for $4 \le x \le y \le 158$: bc is schoolbook multiplication, 22 is Karatsuba's algorithm, 33 is Toom-3, 32 is Toom-(3, 2), 44 is Toom-4, and 42 is Toom-(4, 2). This graph was obtained on a Core 2, with GMP 5.0.0, and GCC 4.4.2. Note that for $x \le (y+3)/4$, only the schoolbook multiplication is available; since we did not consider the algorithm that cuts the larger operand into several pieces, this explains why bc is best for say $x = 32$ and $y = 158$.

Rysunek 1: Algorytm mnożenia stosowany w zależności od długości słów. *Modern Computer Arithmetic*

Następnie zostanie zbadana zależoność czasu potrzebnego do przeprowadzenia operacji mnożenia w zależności od tego jak duże są mnożone liczby dla utworzonej przez nas implementacji.

Następnie te same pary liczb zostaną pomnożone z wykorzystaniem biblioteki GMP (GNU Multiple Precision Arithmetic Library) oraz w języku Python. Umożliwi to wyznaczenie analogicznej zależności jak dla naszej implementacji, oraz ich porównanie.

1.3 Plan osiągnięcia celów projektowych

Szkielet implementacji

- Przygotowanie specyfikacji – opis struktury programu, reprezentacji liczb BigInt oraz wymagań dla algorytmu Toom-3.
- Generowanie wstępnego szkieletu kodu – wykorzystanie modelu Gemini 3 do wygenerowania podstawowej struktury projektu w C++.
- Dokumentacja promptów – zapisywanie wszystkich użytych promptów oraz otrzymanych odpowiedzi (prompt documentation).
- Weryfikacja wygenerowanego kodu – ręczna weryfikacja oraz przygotowanie do dalszej implementacji algorytmów.

Implementacja dodawania BigInt

- Algorytm dodawania – niezbędny do poprawnego działania mnożenia.

Algorytm mnożenia

- Implementacja Toom-3 – wykorzystując wcześniejszy szkielet.

Eksperyment i benchmarki

- Przygotowanie próbek – z użyciem gotowej biblioteki do generowania liczb pseudolosowych i zapis danych wejściowych według ustalonej metodologii.
- Testy własnej implementacji – pomiar czasu mnożenia próbek.
- Testy rozwiązań GMP – benchmark względem profesjonalnego standardu C++.
- Testy rozwiązań wbudowanych w Pythonie – pomiar czasu w Pythonie.
- Zapisanie wyników – względem ustalonej metodologii (.csv).

Analiza wyników

- Wykresy i analiza danych – wykorzystanie bibliotek takich jak scipy, numpy, matplotlib, seaborn, pandas.
- Wnioski końcowe – podsumowanie efektywności algorytmu w praktyce.

2 Generowanie próbki

Próbka jest wygenerowana z użyciem języka Python, konkretnie za pomocą modułu **random** (patrz: *random — Generate pseudo-random numbers* [5]).

Zbiory są wygenerowane jak wspomniano w metodologii. Uruchomiony skrypt: **data_gen.py** tworzy 20 plików tekstowych, każdy przeznaczony liczbom o danej wielkości (ilości słów).

Każda z stu linii jest przeznaczona na jedną liczbę. Są one zapisane w bazie $\beta = 2^{32}$. Każde słowo maszynowe jest rozdzielone znakiem spacji i są one ułożone w kolejności od najbardziej znaczącej do najmniej znaczącej.

Skrypt generuje po kolei każde słowo za pomocą metody **.getrandbits(k)**, gdzie nasze $k = 32$. Metoda ta korzysta głównie z generatora Mersenne'a Twister'a, który generuje liczby pseudolosowe o rozkładzie jednostajnym (patrz: "Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudorandom Number Generator" [3]).

3 Implementacja w języku Python

Język Python posiada natywną obsługę liczb całkowitych o dowolnej precyzji (*arbitrary-precision integers*), co eliminuje konieczność stosowania zewnętrznych bibliotek. Rozmiar reprezentowanych liczb nie posiada sztywnego ograniczenia technicznego [4]; jedynym czynnikiem limitującym jest dostępna pamięć operacyjna komputera.

Skrypt **main_PYTHON.py** składa się z zestawu funkcji pomocniczych oraz funkcji głównej *main_func*, odpowiedzialnej za przeprowadzenie operacji mnożenia oraz pomiar czasu ich wykonania.

3.1 vector_to_int

Funkcja pomocnicza służąca do konwersji wektora 32-bitowych słów maszynowych na typ `int` języka Python. Jako argument przyjmuje wektor słów, a zwraca obiekt o nieograniczonej precyzji. Proces rekonstrukcji liczby opiera się na przesunięciu bitowym w lewo (*bit-shift*) oraz sumowaniu kolejnych fragmentów danych przy użyciu operatora alternatywy bitowej (*OR*).

3.2 choose_time_unit

Funkcja odpowiedzialna za automatyczny dobór jednostki czasu oraz odpowiednie skalowanie wartości liczbowej w celu poprawy czytelności wyników. Przyjmuje czas trwania operacji w mikrosekundach i zwraca krotkę *(wartość, jednostka)*, gdzie wartość jest typu `float`, a jednostka typu `string`.

3.3 main_func

Główna procedura sterująca procesem obliczeniowym. Realizuje ona mnożenie par liczb wczytanych z pliku, a następnie zapisuje czas egzekucji w pliku wynikowym. Funkcja przyjmuje dwie ścieżki do plików: wejściową oraz wyjściową.

Zgodnie z przyjętą metodologią, plik wejściowy musi zawierać parzystą liczbę rekordów (po jednym w każdym wierszu), reprezentujących liczby o podstawie $\beta = 2^{32}$. Funkcja wczytuje dane parami, dokonuje ich konwersji na typ `int`, a po wykonaniu mnożenia i sformatowaniu jednostek czasu, zapisuje wynik w formacie:

liczba słów maszynowych, czas (z precyzją do dwóch miejsc po przecinku), jednostka

4 Implementacja z wykorzystaniem biblioteki GNU MP

Biblioteka GNU MP (GMP) jest wysokowydajnym narzędziem napisanym w języku C, przeznaczonym do wykonywania operacji arytmetycznych na liczbach o dowolnej precyzji. W niniejszym projekcie wykorzystano wyłącznie moduł operacji na liczbach całkowitych (`mpz_t`). Wykorzystana implementacja pochodzi z oficjalnej dystrybucji i nie została poddana modyfikacjom.

4.1 vector_to_mpz

Funkcja pomocnicza dokonująca transformacji wektora typu `std::vector<uint32_t>` na obiekt struktury `mpz_t`. Jej zadaniem jest ułatwienie procesu importu danych i poprawa czytelności kodu źródłowego. Funkcja nie zwraca wartości (`void`), lecz modyfikuje przekazany przez referencję obiekt `mpz_t` za pomocą funkcji `mpz_import`, zgodnie ze specyfikacją dokumentacji GMP[2].

4.2 TimeResult

W celu ustrukturyzowania danych pomiarowych zdefiniowano strukturę `TimeResult`, zawierającą dwa pola: `double value` (wartość liczbowa) oraz `std::string unit` (oznaczenie jednostki). Ułatwia ona zarządzanie wynikami pomiarów i ich późniejszą prezentację.

4.3 choose_time_unit

Funkcja analogiczna do wersji Pythonowskiej, dokonująca konwersji surowego pomiaru czasu (wyrażonego w nanosekundach) na obiekt struktury `TimeResult`.

4.4 main_func

Główna funkcja odpowiedzialna za wczytywanie danych, realizację mnożenia oraz zapisanie wyników biblioteki GMP. Przyjmuje nazwy plików wejściowego i wyjściowego.

Podobnie jak w przypadku skryptu Python, wymagany jest plik z parzystą liczbą wierszy zawierających liczby o podstawie $\beta = 2^{32}$. Funkcja importuje dane do formatu `mpz_t`, wykonuje mnożenie, a następnie zapisuje parametry eksperymentu do pliku wyjściowego w formacie identycznym jak w przypadku implementacji w Pythonie.

Funkcja zwraca kod błędu `1` w przypadku niepowodzenia dostępu do plików lub `0` po pomyślnym zakończeniu wszystkich procedur obliczeniowych.

5 Analiza wyników

Zebrane dane zostały poddane analizie w celu zbadania zależności czasu potrzebnego na wykonanie mnożenia od rozmiaru mnożonych liczb oraz użytego narzędzia. Do uruchomienia wszystkich napisanych skryptów w odpowiedniej kolejności użyto narzędzia CMake. Za przygotowanie do analizy oraz samą analizę otrzymanych wyników odpowiada skrypt napisany w języku Python o nazwie **analysis.py**. Zawiera on funkcje opisane poniżej.

5.1 load_file

Funkcja pobiera zawartość wybranego pliku ze zmierzonymi czasami potrzebnymi na wykonanie operacji mnożenia. Funkcja zwraca obiekt ramki danych zawierający dane przeznaczone do analizy. Funkcja ta zgłasza błąd w przypadku gdy szukany plik nie istnieje.

5.2 equalize_units

Funkcja ujednolica jednostki, w których zapisane są czasy w wybranej ramce danych.

5.3 remove_outliers

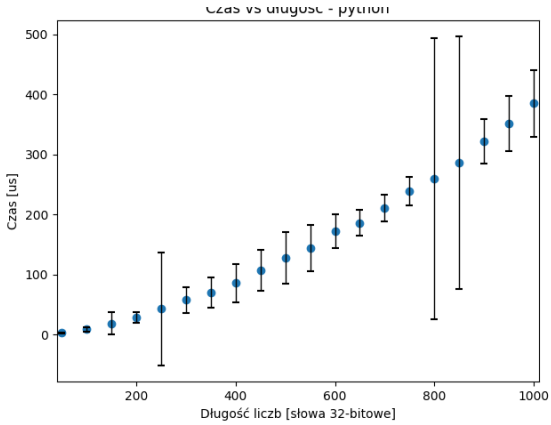
Funkcja ta usuwa wartości odstające w zbiorze czasów uzyskanych dla liczb o różnych długościach. Reguła, którą zastosowano do odrzucenia wartości odstających to tzw. reguła trzech σ, polegająca na odrzucenia wartości różniących się o więcej niż trzy odchylenia standardowe od średniej.

5.4 calculate_stats

Funkcja ta liczy czas średni oraz odchylenie standardowe w zbiorze otrzymanych czasów dla liczb o różnej długości.

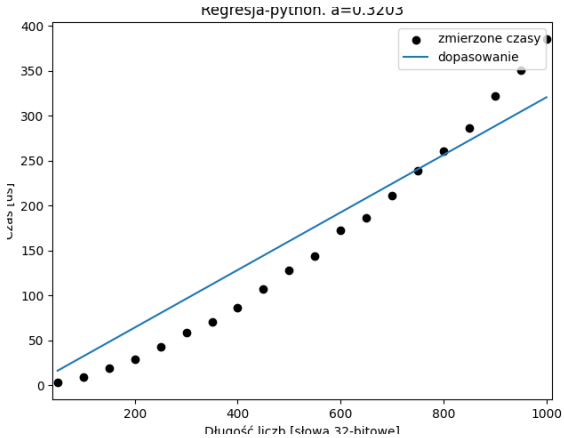
5.5 plot_one_method

Funkcja ta generuje wykres dla wybranej metody. Oś pozioma przedstawia rozmiar liczb w słowach 32-bitowych, a oś pionowa czas w mikrosekundach. Średnie czasy przedstawione są za pomocą punktów. Odchylenia sytandardowe przedsatwione są za pomocą punktów. Przykładowy wykres dla mnożenia wykonanego w Pythonie został przedstawiony poniżej.



5.6 draw_regression

Funkcja ta dopasowuję regresję nieliniową o równaniu zdefiniowanym w funkcji **model** do uzyskanych czasów. W kolejnym kroku generowany jest odpowiedni wykres. Przykładowy wykres dla mnożenia wykonanego w Pythonie został przedstawiony poniżej, tutaj dla przypadku gdy dopasowywana funkcja jest funkcją liniową.



5.7 workflow

Funkcja ta odpowiada za wywołanie funkcji przedstawionych powyżej w odpowiedniej kolejności.

6 Cel dodatkowy

W ramach rozszerzenia projektu podjęto próbę wyznaczenia granic wydajnościowych współczesnych rozwiązań w zakresie arytmetyki komputerowej. Głównym celem tej części badania było określenie rzędu wielkości liczb (wyrażonego w liczbie słów maszynowych), których operacja mnożenia mieści się w zadanym przybliżeniu czasowym: 1 milisekundy oraz 1 sekundy.

Do przeprowadzenia testów wykorzystano bibliotekę GNU MP (GMP), z racji najefektywniejszej implementacji. Należy zaznaczyć, że wyniki są ściśle związane z architekturą sprzętową komputera, na którym przeprowadzony jest eksperyment. Pomiary przeprowadzono na jednostce wyposażonej w procesor Intel Core i5-7400 oraz 8,0 GiB pamięci RAM.

Na podstawie wstępnej empirycznej analizy ustalono, że:

- Granica czasu wykonania wynosząca 1 ms odpowiada liczbom o długości od 8 700 do 9 000 limbów (słów 32-bitowych).
- Granica czasu wykonania wynosząca 1 s odpowiada liczbom o długości od 3 650 000 do 3 700 000 limbów.

6.1 Generowanie danych testowych

W celu weryfikacji postawionych hipotez opracowano skrypt w języku Python, wykorzystujący mechanizmy opisane w rozdziale 2. Wygenerowano dwa zestawy danych testowych:

1. 7 par liczb o długości od 8 700 do 9 000 limbów z krokiem wynoszącym 50 słów.
2. 6 par liczb o długości od 3 650 000 do 3 700 000 limbów z krokiem wynoszącym 10 000 słów.

6.2 Wyniki eksperymentu

Dane poddano operacji mnożenia z wykorzystaniem implementacji opartej na bibliotece GMP (opisanej w rozdziale 4). Uzyskane wyniki prezentują się następująco:

Liczba słów	**Czas wykonania**	**Jednostka**
8 700	1,02190	ms
8 750	1,06987	ms
8 800	1,03674	ms
8 850	984,155	μs
8 900	972,745	μs
8 950	1,01806	ms
9 000	987,652	μs
3 650 000	977,098	ms
3 660 000	990,404	ms
3 670 000	1,04706	s
3 680 000	1,05475	s
3 690 000	1,04270	s
3 700 000	1,02102	s

Tabela 1: Wyniki pomiarów czasu mnożenia dla wybranych rzędów wielkości.

Aby zobrazować skalę problemu obliczeniowego, wyznaczono teoretyczną liczbę cyfr dziesiętnych dla rekordu o długości 3 660 000 słów maszynowych. Przyjmując, że każde słowo składa się z 32 bitów, całkowita liczba bitów B wynosi:

$$B = 3\,660\,000 \times 32 = 117\,120\,000 \text{ bitów}$$

Liczbę cyfr dziesiętnych D wyznacza się, korzystając z własności logarytmów. Dla liczby x, liczba jej cyfr w systemie o podstawie 10 jest równa $\lfloor \log_{10}(x) \rfloor + 1$. W przybliżeniu można przyjąć zależność:

$$\begin{aligned} D &\approx B \times \log_{10}(2) \\ D &\approx 117\,120\,000 \times 0,301029995 \\ D &\approx 35\,256\,633 \end{aligned}$$

Z powyższych obliczeń wynika, że badana liczba posiada ponad **35 milionów cyfr dziesiętnych**.

Przyjmując założenie o szerokości pojedynczej cyfry równej 0,5 cm, zapisanie tej wartości w jednej linii wymagałoby płaszczyzny o długości:

$$L = 35\,256\,633 \times 0,005 \text{ m} \approx 176\,283 \text{ m} \approx 176,3 \text{ km}$$

Tyle ile wynosi mniej więcej odległośc między Gdańskiem a Toruniem.

Fakt, że biblioteka GMP jest w stanie wykonać operację mnożenia dwóch tak ogromnych obiektów w czasie poniżej jednej sekundy (990, 404 ms), świadczy o niezwykle wysokiej efektywności zaimplementowanych algorytmów (prawdopodobnie Szybkiej Transformaty Fouriera - FFT) oraz współczesnych jednostek CPU.

References

[1] Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*. Vol. 18. Cambridge Monographs on Applied and Computational Mathematics. Cambridge: Cambridge University Press, 2010. ISBN: 978-0-521-19469-3.

[2] Torbjörn Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. Dostęp online. 2023. URL: `https://gmplib.org/manual/`.

[3] Makoto Matsumoto and Takuji Nishimura. "Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudorandom Number Generator". In: *ACM Transactions on Modeling and Computer Simulation* 8.1 (1998), pp. 3–30. DOI: `10.1145/272991.272995`.

[4] Python Software Foundation. *Integer Objects*. Online; accessed 2026. Python Software Foundation. URL: `https://docs.python.org/3/c-api/long.html`.

[5] Python Software Foundation. *random — Generate pseudo-random numbers*. Online; accessed 2026. Python Software Foundation. 2024. URL: `https://docs.python.org/3/library/random.html`.